RELATÓRIO TÉCNICO

KernelLab: Simulador de Gerenciamento de CPU, Memória e Dispositivos

Aplicação dos conceitos de Sistemas Operacionais ao cenário da MedControl Systems

Aluno: Osni Mendes Calixto
Disciplina: Sistemas Operacionais
Instituição: UniFECAF
Engenharia da Computação

2026

Resumo

Este relatório apresenta a fundamentação teórica do KernelLab, um simulador didático que reproduz decisões típicas de um sistema operacional: escalonamento da CPU, cálculo de métricas, gerenciamento de memória virtual, controle de dispositivos e proteção de recursos compartilhados. O estudo utiliza como contexto a MedControl Systems, uma plataforma hipotética de monitoramento de equipamentos médicos, na qual tarefas como monitoramento cardíaco, transmissão de telemetria, geração de relatórios, emissão de alarmes e cópias de segurança disputam recursos limitados. O foco não é construir um sistema operacional completo, mas comparar políticas e demonstrar como escolhas de projeto afetam tempo de resposta, espera, justiça, falhas de página e segurança.

1. Contextualização do problema

A MedControl Systems precisa processar atividades com criticidades diferentes. Um alarme clínico deve ser atendido rapidamente; a transmissão de telemetria precisa manter regularidade; relatórios e backups podem tolerar maior demora. Se todas as tarefas forem tratadas de maneira idêntica, uma atividade longa pode atrasar eventos urgentes, enquanto tarefas de baixa prioridade podem ficar esperando por tempo excessivo. Além disso, processos concorrem por memória, rede, disco e áudio, criando disputas e riscos de acesso indevido.

O KernelLab permite experimentar políticas sob condições controladas. Para tornar a comparação clara, considera-se uma CPU única, processos com instante de chegada e duração de CPU conhecidos e recursos de memória e dispositivos definidos na entrada. Os resultados são uma aproximação didática: não substituem validação clínica, testes de tempo real nem análise de segurança de um produto médico real.

O cenário inclui, entre outras, as tarefas P1 (monitoramento cardíaco), P2 (telemetria), P3 (relatórios), P4 (alarme crítico), P5 (backup) e P6 (atualização de dados). A P4 possui prioridade elevada por sua função crítica. A simulação deve tornar visível o custo de cada política, em vez de presumir que um único algoritmo seja melhor em todas as situações.

2. Estados e ciclo de vida dos processos

Um processo é um programa em execução, acompanhado de contexto, estado, recursos e informações de controle. Em um modelo didático, seus estados principais são:

- **Novo:** o processo está sendo criado e ainda não disputa a CPU.
- **Pronto:** está apto a executar e aguarda a CPU.
- **Executando:** está usando a CPU.
- **Bloqueado/esperando:** aguarda um evento, como conclusão de uma operação de disco ou rede.
- **Finalizado:** terminou ou foi encerrado.

As transições ocorrem por admissão, despacho para a CPU, preempção, solicitação/conclusão de E/S e término. Em um sistema preemptivo, o escalonador pode retirar a CPU de uma tarefa para executar outra, por exemplo quando chega uma tarefa de prioridade maior ou termina a fatia de tempo do Round Robin. No KernelLab, essas transições devem ser registradas na linha do tempo para explicar por que um processo esperou ou ficou bloqueado.

Aplicação à MedControl: se P4 (alarme crítico) chegar enquanto P3 (relatórios) estiver executando, uma política preemptiva por prioridade pode interromper P3 e atender P4. Em contrapartida, se P5 (backup) ficar sempre atrás de tarefas prioritárias, poderá sofrer inanição (starvation). A simulação deve registrar esse efeito, não apenas o tempo total de execução.

3. Algoritmos de escalonamento da CPU

O escalonador seleciona qual processo pronto recebe a CPU. Para este relatório são considerados quatro algoritmos clássicos: FCFS/FIFO, SJF, Prioridade e Round Robin. Se a implementação final do KernelLab

utilizar variantes diferentes, os nomes e a análise desta seção devem ser ajustados para corresponder exatamente ao código.

3.1 FCFS (First-Come, First-Served)

Executa os processos pela ordem de chegada e normalmente não preempta a tarefa em execução. É simples e tem baixo custo de decisão, mas pode apresentar o efeito comboio: um processo longo na frente atrasa processos curtos e urgentes. Na MedControl, um relatório ou backup longo poderia atrasar o alarme P4 se chegasse antes dele. Portanto, FCFS é uma boa linha de base, mas não oferece garantia de resposta rápida para eventos críticos.

3.2 SJF (Shortest Job First)

Seleciona o processo pronto com menor duração estimada de CPU. Em sua forma não preemptiva, a tarefa escolhida executa até terminar ou bloquear; a variante preemptiva é conhecida como SRTF (Shortest Remaining Time First). SJF pode reduzir a espera média quando as durações são conhecidas, mas depende de estimativas corretas e pode deixar tarefas longas esperando repetidamente. No cenário, tarefas curtas podem concluir depressa, mas o alarme não deve depender apenas de seu burst ser curto: sua criticidade também importa.

3.3 Escalonamento por prioridade

A cada decisão, escolhe o processo pronto com maior prioridade segundo a convenção definida no simulador. Neste relatório, assume-se que um número menor representa prioridade mais alta, de modo que P4, com prioridade 0, precede tarefas de prioridade inferior. A política pode ser preemptiva ou não preemptiva; essa escolha deve ser explicitada no programa e no relatório de resultados. O risco é a inanição de tarefas de baixa prioridade. Uma solução clássica é o envelhecimento (aging), que aumenta gradualmente a prioridade de processos que esperam por muito tempo.

3.4 Round Robin (RR)

Distribui a CPU em fatias de tempo chamadas quantum. Ao fim da fatia, se o processo não terminou, ele retorna à fila de prontos. A política preemptiva tende a melhorar a capacidade de resposta e a justiça entre processos interativos. Um quantum muito grande aproxima o comportamento de FCFS; um quantum muito pequeno aumenta trocas de contexto e o custo de escalonamento. Para a MedControl, RR ajuda a evitar que uma única tarefa monopolize a CPU, mas, sem mecanismo adicional de prioridade, não garante atendimento imediato do alarme crítico.

Comparação resumida

Algoritmo	Vantagem	Limitação	Efeito esperado no cenário
FCFS	Simples e previsível pela chegada	Efeito comboio; sem resposta urgente garantida	P4 pode esperar atrás de tarefas longas
SJF	Pode reduzir espera média	Exige estimar duração; tarefas longas podem esperar	Relatórios curtos podem terminar cedo
Prioridade	Favorece tarefas críticas	Risco de inanição das prioridades baixas	P4 é favorecida; P5 precisa de proteção contra espera infinita
Round Robin	Compartilha CPU com justiça	Quantum influencia resposta e overhead	Evita monopólio, mas pode atrasar alarmes sem prioridade

4. Conceitos e métricas de desempenho

A comparação dos algoritmos precisa usar métricas calculadas da mesma forma para todos os cenários. Para cada processo, definem-se chegada (A), início da primeira execução (S), término (C) e duração de CPU (B).

• **Tempo de retorno (turnaround):** C − A. Mede o tempo total entre chegada e conclusão.
• **Tempo de espera:** tempo acumulado em que o processo permanece pronto, aguardando CPU. Em um caso simples sem bloqueios, pode ser calculado como retorno − burst; com E/S e múltiplos bloqueios, deve ser contabilizado pela linha do tempo.
• **Tempo de resposta:** S − A. Mede quanto tempo decorre até a primeira execução, importante para alarmes.
• **Utilização da CPU:** proporção do período em que a CPU está ocupada.
• **Throughput:** quantidade de processos concluídos por unidade de tempo.
• **Trocas de contexto:** mudanças entre processos, que representam custo operacional.

No cenário MedControl, a média sozinha não basta. O tempo de resposta de P4 deve ser observado individualmente, pois um bom valor médio pode esconder um atraso inaceitável em um evento crítico. Também é útil registrar o maior tempo de espera, a quantidade de falhas de página e o tempo de bloqueio por dispositivo. O relatório de resultados deve informar as hipóteses utilizadas, como preempção, quantum, política de desempate e duração dos acessos a dispositivos.

5. Funcionamento da memória virtual

Memória virtual permite que cada processo opere em um espaço de endereçamento lógico, separado dos endereços físicos da RAM. Em sistemas paginados, o espaço é dividido em páginas e a memória física em quadros (frames). Uma tabela de páginas mapeia páginas virtuais para quadros físicos; a TLB pode acelerar traduções frequentes. Se uma página necessária não estiver residente, ocorre uma falha de página (page fault), que exige intervenção do sistema operacional. A página pode ser carregada do armazenamento e, quando a RAM está cheia, outra página pode precisar ser substituída.

O KernelLab pode representar cada processo por uma necessidade de memória em MB e uma capacidade total de RAM configurada. Caso o total demandado exceda a capacidade, o simulador deve explicitar a política adotada para decidir quais páginas saem e quais entram. Uma simplificação deve ser declarada: MB solicitados por processo não equivalem, por si só, a uma simulação completa de paginação; para contar falhas de página com fidelidade, é necessário definir tamanho de página e sequência de referências a páginas.

No cenário, P5 (backup) solicita 300 MB e P3 (relatórios), 200 MB. Se essas demandas competirem com tarefas de monitoramento, a falta de memória pode aumentar E/S e atrasar o processamento. A política de memória precisa evitar que uma atividade secundária provoque pressão excessiva sobre a memória usada por tarefas críticas. A simulação didática não deve afirmar que reproduz integralmente a memória de um sistema operacional real sem modelar referências e eventos de página.

6. Substituição de páginas: FIFO, LRU e Optimal

Quando não há quadro livre, um algoritmo de substituição escolhe a página vítima. O resultado depende da sequência de referências e do número de quadros disponíveis.

6.1 FIFO (First-In, First-Out)

Remove a página que está há mais tempo na memória, independentemente de quantas vezes foi usada recentemente. É simples de implementar, mas pode retirar uma página muito ativa. Em determinadas sequências, FIFO pode apresentar a anomalia de Belady: aumentar o número de quadros não necessariamente reduz as falhas.

6.2 LRU (Least Recently Used)

Substitui a página cujo uso ocorreu há mais tempo. Usa a recência como aproximação da probabilidade de reutilização, aproveitando a localidade temporal. Geralmente é uma escolha prática melhor que FIFO em muitos padrões, mas exige acompanhar a ordem ou o instante do último uso, o que tem custo de implementação.

6.3 Optimal (OPT)

Substitui a página cuja próxima utilização ocorrerá mais distante no futuro, ou que não será utilizada novamente. É um algoritmo teórico porque exige conhecer antecipadamente as referências futuras. Serve como referência para comparar quantas falhas FIFO e LRU produzem, mas não é uma política diretamente implementável em execução real sem conhecimento do futuro.

Para comparar corretamente, o KernelLab deve usar a mesma sequência de referências e o mesmo número de quadros em cada algoritmo. Deve contar falhas de página e mostrar a página removida em cada ocorrência. Se o simulador ainda não recebe sequências de referências, essa parte deve ser apresentada como modelo simplificado ou ampliada com uma lista de páginas acessadas por processo.

7. Concorrência, mutex e semáforos

Concorrência ocorre quando várias tarefas progridem em períodos sobrepostos e disputam recursos. Mesmo com uma CPU única, há concorrência lógica: processos alternam entre CPU e espera por E/S. Se mais de uma tarefa puder acessar um recurso compartilhado, operações podem interferir umas nas outras e produzir condições de corrida. Uma seção crítica é o trecho que acessa ou altera estado compartilhado e exige coordenação.

Um **mutex** oferece exclusão mútua: em geral, somente um processo ou thread por vez pode possuir o bloqueio e acessar a seção crítica protegida. Um **semáforo** mantém um contador de permissões e pode controlar o acesso a um conjunto limitado de recursos; um semáforo binário pode ter uso semelhante ao de um mutex, embora seus contratos e regras de propriedade não sejam necessariamente iguais.

Aplicação à MedControl: o registro de alarmes deve evitar que duas tarefas corrompam a mesma estrutura de dados; o acesso ao dispositivo de áudio pode ser serializado por mutex ou controlado por um semáforo de capacidade; e a fila de transmissão pode ter acesso coordenado entre produtores e consumidores. A implementação precisa definir quem adquire e libera cada recurso e como lida com falhas. Também deve considerar deadlock, starvation e inversão de prioridade. Para sistemas críticos, técnicas como herança de prioridade ou protocolos de teto de prioridade podem reduzir o risco de uma tarefa de baixa prioridade bloquear indiretamente uma tarefa urgente.

8. Segurança no acesso aos dispositivos

Dispositivos como rede, disco e áudio são recursos compartilhados e devem ser acessados por meio de interfaces controladas. O sistema precisa validar se o processo tem autorização para usar o dispositivo e quais operações pode realizar. A separação de privilégios, a validação de parâmetros, o controle de filas, os limites de uso e o registro de eventos reduzem o risco de acesso indevido, corrupção de dados ou monopolização de recursos.

No KernelLab, cada processo deve declarar os dispositivos necessários e o simulador deve verificar se o pedido está autorizado antes de conceder acesso. Se dois processos solicitarem um dispositivo exclusivo, o segundo deve aguardar a liberação. Para dispositivos compartilháveis, como rede, a política pode permitir concorrência controlada, limitada por capacidade. Um processo não autorizado não deve adquirir o recurso apenas por estar em primeiro lugar na fila. Os eventos de acesso permitido, negado, bloqueado e liberado devem ser rastreáveis.

Para a MedControl, essa regra é especialmente relevante para o caminho do alarme: uma tarefa comum não deve impedir indefinidamente a emissão do sinal sonoro, e uma rotina de backup não deve ocupar recursos sem limites. O simulador demonstra princípios de segurança e coordenação, mas não constitui certificação de software médico nem substitui requisitos regulatórios, análise de risco e testes de hardware.

9. Relação entre teoria e decisões implementadas

A utilidade do KernelLab está em relacionar cada conceito a uma decisão observável e a uma métrica. A tabela abaixo propõe uma correspondência para orientar a implementação e a análise dos resultados.

Conceito	Decisão no simulador	Evidência a registrar
Escalonamento	Selecionar FCFS, SJF, prioridade ou RR; definir preempção e quantum	Ordem de execução, resposta, espera, retorno e trocas de contexto
Criticidade	Dar prioridade alta ao processo de alarme P4	Tempo de resposta de P4 e comparação entre políticas
Justiça	Aplicar aging ou limite de espera, se previsto	Maior espera de P5 e número de tarefas concluídas
Memória virtual	Definir RAM, tamanho de página e sequência de referências	Falhas de página e páginas substituídas
Concorrência	Proteger filas/estruturas compartilhadas e controlar dispositivos	Bloqueios, liberações e conflitos evitados
Segurança	Validar permissões antes de conceder dispositivo	Pedidos autorizados/negados e motivo

A análise deve apresentar resultados reais obtidos ao executar o programa. Não se deve preencher valores de métricas por suposição. Recomenda-se executar o mesmo conjunto de processos com cada algoritmo e comparar: (1) resposta de P4; (2) espera média e máxima; (3) retorno médio; (4) utilização da CPU; (5) falhas de página; (6) tempo de bloqueio por dispositivo; e (7) tentativas de acesso não autorizado. Se um algoritmo melhora uma métrica e piora outra, essa troca deve ser discutida explicitamente.

10. Conclusão

O estudo mostra que o gerenciamento de CPU, memória e dispositivos exige equilibrar rapidez, justiça, eficiência e segurança. FCFS é simples, SJF pode reduzir espera média, prioridade favorece tarefas críticas e Round Robin distribui o tempo de CPU. Nenhum deles, isoladamente, resolve todos os requisitos do cenário MedControl. O atendimento do alarme depende de uma política de prioridade bem definida e de recursos não ficarem bloqueados por tarefas secundárias; ao mesmo tempo, aging ou limites de espera podem reduzir a inanição de processos como backup.

Na memória, FIFO e LRU são políticas implementáveis com compromissos diferentes, enquanto Optimal funciona como referência teórica. Mutexes e semáforos ajudam a coordenar o acesso concorrente, e a verificação de permissões protege dispositivos compartilhados. A contribuição do KernelLab é permitir que essas decisões sejam simuladas e comparadas com métricas. A conclusão final do projeto deve ser complementada pelos dados gerados pelo próprio programa, destacando as limitações do modelo e evitando alegações de segurança ou tempo real que não tenham sido demonstradas.

11. Referências

SILBERSCHATZ, Abraham; GALVIN, Peter B.; GAGNE, Greg. *Operating System Concepts*. 10. ed. Hoboken: Wiley, 2018.

TANENBAUM, Andrew S.; BOS, Herbert. *Modern Operating Systems*. 4. ed. Boston: Pearson, 2015.

STALLINGS, William. *Operating Systems: Internals and Design Principles*. 9. ed. Hoboken: Pearson, 2018.

IEEE. *IEEE Standard for System, Software, and Hardware Verification and Validation*. IEEE Std 1012-2016. Piscataway: IEEE, 2016.

PYTHON SOFTWARE FOUNDATION. *Python 3 Documentation*. Disponível em: https://docs.python.org/3/. Acesso em: 13 out. 2026.

Observação: as obras de sistemas operacionais fundamentam os conceitos gerais. A descrição específica da MedControl e os exemplos de processos são premissas do projeto KernelLab; os resultados quantitativos devem ser obtidos pela execução da implementação.